

WEEK 6 SKILL OSSP

Part 1 — Backslash Escaping / Special Character Processing

1. Created a new folder for week 6 skill

```
samanth@DESKTOP-BLT5L3R:~$ mkdir week6_s
samanth@DESKTOP-BLT5L3R:~$ cd ~/week6_s
samanth@DESKTOP-BLT5L3R:~/week6_s$ pwd
/home/samanth/week6_s
```

2. Basic Backslash Escaping

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ echo "Hello\nWorld"
echo -e "Hello\nWorld"
Hello\nWorld
Hello
World
```

3. Escaped Spaces

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ echo "Hello\nWorld"
echo -e "Hello\nWorld"
Hello\nWorld
Hello
World
```

4. Escape Special Characters

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ echo \*
echo \&
echo \|
*
&
|
```

5. Preserve Characters

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ echo '$HOME'
echo "$HOME"
$HOME
/home/samanth-ubuntu
```

6. Validate Parser Output

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ printf '<%s>\n' Hello\ World
printf '<%s>\n' "Hello World"
printf '<%s>\n' 'Hello World'
<Hello World>
<Hello World>
<Hello World>
```

1. Text complex input

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ echo "Hello\ World \$USER \& \*"
Hello\ World $USER \& \*
```

Summary:

The experiment demonstrates how backslash escaping is used in Linux shell processing to preserve spaces and special characters. Escape sequences such as `\n` were tested, along with escaped spaces, shell metacharacters, quoted strings, and complex inputs.

The outputs were validated using `printf` to observe how the shell parses individual arguments. The experiment confirms that escaping and quoting can control how the shell interprets special characters and preserves intended input.

Part 2 — Child Processes

1. Creating the Child Process Program, Compiling and Execute

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ nano child_process.c
samanth-ubuntu@samanth-ubuntu:~/week6_s$ gcc child_process.c -o child_process
./child_process
Parent Process: PID = 6809
Child Process: PID = 6810
Waiting for child process...
Executing command: ls
child_process child_process.c
child process completed.
```

2. Passing Arguments to a Child Process

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ nano child_process.c
samanth-ubuntu@samanth-ubuntu:~/week6_s$ gcc child_process.c -o child_process
./child_process
Parent Process: PID = 6867
Child Process: PID = 6868
Running: echo Hello OSSP Week 6
Hello OSSP Week 6
Child process completed.
```

3. Final Process Check

```
samanth-ubuntu@samanth-ubuntu:~/week6_s$ ps
  PID TTY          TIME CMD
 5178 pts/0    00:00:00 bash
 6901 pts/0    00:00:00 ps
```

Summary:

The experiment demonstrated backslash escaping and special character processing in Ubuntu. We tested escaped spaces, special symbols, escape sequences, character preservation using quotes, and validated how the shell parses normal and complex inputs using `printf`.

We also implemented process creation and execution in C using `fork()`, `exec()`, and `wait()`. The child process executed commands with arguments, the parent process managed the child, and execution errors were handled using `perror()`.